

LRU File Block Cache: Design & Implementation

■ The Goal

Design and implement a simple Least Recently Used (LRU) caching mechanism for file blocks as a response to a personal coding challenge.

■■ Disclaimers

- **Human-Written:** This document is hand-written. For an AI-assisted version with automated spell-checks, see *README-AI.md*.
- **Licensing:** See *LICENSE.md* for full terms.
- **Credits:** Original work inspired by existing open-source software. Individual subsystem licenses apply.
- **Warranty:** Provided "as-is" without any explicit or implied guarantees. Use at your own risk.
- **Alternative Solutions:** This is a conceptual demonstration. Conduct your own Cost-Benefit Analysis (CBA) before production use.

■ Sub-Goals

- Keep it light-weight and simple.
- Keep it as generic as possible.
- Make it easy for interns or students to read.

■ Context

Filesystems and document databases operate at the file level. Applications can achieve massive performance gains by caching specific file blocks during active I/O sessions.

■■ Technical Considerations

Cache Tagging

- **Global Context:** Combine File Block Number (FBN) with the Inode number to persist cache tags across sessions.
- **Transient Context:** Combine FBN with the File Descriptor (FD) for temporary, session-based caching.
- **LBA Alternative:** Use the Logical Block Address (LBA), which requires a unique volume identifier (like a UUID) to prevent conflicts.

Fetching vs. Deriving FBN

- **Fetching:** Traverses inodes directly. This scales well but adds high complexity, so it is skipped for this project.
- **Deriving:** Calculated mathematically from the file offset: $(\text{offset} / \text{FILE_BLOCK_SIZE})$.

File Block Size Dynamics

- Typical personal computing block sizes range from 1KB to 4KB (usually default to 4KB).
- Large file servers may scale up to 1MB+ blocks.
- Uniformity can be enforced by setting `FILE_BLOCK_SIZE` manually.

Out of Scope

- **Custom Application Tags:** Internal UUIDs or application-specific references are skipped.
- **Sparse Files:** Empty blocks in sparse files can waste cache lines; handling this is left as a future enhancement.
- **Orphan Control:** Automated scanning and clearing of orphaned blocks is excluded.
- **Session Management:** Saving and restoring cache sessions is excluded.

■■ Design Decisions

Filesystem caching is complex. This implementation intentionally prioritises conceptual clarity over production-grade edge-case handling.

1. Cache Line Look-Up

Doubly-Linked Lists: Simple to implement and test, but has linear time complexity $O(n)$. It destroys CPU cache lines and triggers random RAM reads at scale. Unsuitable for more than a few hundred blocks.

Hash Tables: Ideal constant time complexity $O(1)$. However, it requires extremely high memory consumption (e.g., a 32-bit hash table can require up to 16GB of RAM).

Binary Trees (Selected): Highly balanced performance using self-balancing structures like Red-Black Trees. Logarithmic time complexity $O(\log n)$. Uses ready-made standard library structures (`std::set` or `std::map`).

2. LRU Chain Management

To manage expiration tracking, we selected a **hybrid approach**. Real-world systems use an array/vector for recent hits, synchronising the main LRU tree periodically to bypass heavy tree rebalancing on every read operation.

■ Locks and Race Conditions

Concurrency is a deep problem space. This design requires strict locking mechanisms (or lock-free algorithms) across both the trees and linked lists to maintain thread safety and data consistency.